```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Scanner;

public class InputOutputNotes {
    public static void main(String[] args) {

        // Setting up the Scanner
        Scanner in = new Scanner(System.in); // Creating scanner object



        // 1. 1D Arrays (The Line)
        /*
        Concept: A single row of fixed memory slots.

        // INPUT SYNTAX

        Rule: You *must* use standard for loop. *Why?* You need the index(i) to tell java exactly which
        slot (arr[i]) to fill. You CANNOT use an enhanced for-loop for input because it gives you a copy
        of the value, not the slot itself.
         */

        // Step 1: Create the array (must specify size)
        int[] arr = new int[5];

        // Step 2: Loop to read input
        for (int i = 0; i < arr.length; i++) {
            arr[i] = in.nextInt(); // Store input at index 'i'
        }

        // OUTPUT SYNTAX

        /*
        Option A: Enhanced For-loop (Cleanest) Use this when you just want to read values
        and do not care about the index number.
         */
        for (int num : arr) {
            System.out.println(num + " "); //Prints value
        }

        /*
        Option B: The "Lazy" Debug Way. Best for quickly checking your work in an interview.
         */
        System.out.println(Arrays.toString(arr)); //Prints "[1, 2, 3, 4, 5]"
```

```java


        // 2D Arrays (The Grid)
        /*
        Concept: An array where every slot holds another array. You need two loops (Nested loops).
         */

        // INPUT SYNTAX

        // Rule: Outer loop for Rows, Inner loop for Columns.


        // Step 1: Create the matrix (3 rows, 3 columns)
        int[][] arr2D = new int[3][3];

        // Step 2: Nested Loop for Input
        for (int row = 0; row < arr2D.length; row++) {          // 1. Pick a row
            for (int col = 0; col < arr2D[row].length; col++) { // 2. Go through every column in that row
                arr2D[row][col] = in.nextInt();                 // 3. Fill the cell
            }
        }

        // OUTPUT SYNTAX

        /*
        Option A: Enhanced For-loop (Nested) This is elegant but can be tricky to remember.
        Read it as: "For every row (which is an int array) inside the matrix..."
         */
        for (int[] row : arr2D) {               // Get the row array
        for (int col : row) {                   // Get the number inside the row
                System.out.println(col + " ");
            }
            System.out.println();               // New line after each row
        }

        /*
        Option B: The "Lazy" Mix: You can use Arrays.toString() on each row to save time.
         */
        for (int[] row : arr2D) {
            System.out.println(Arrays.toString(arr2D));
        }
```

```java
92
93
94
95
96          // ArrayList (The Dynamic List)
97          /*
98          Concept: A flexible list that grows on its own. You do not use square brackets [].
99          You use methods like .add() and .get()
100          */
101
102         // INPUT SYNTAX
103
104         // Creating a 1D ArrayList
105         ArrayList<Integer> list = new ArrayList<>(5);
106
107         // Case A: You know how many items (Standard Loop)
108         for (int i = 0; i < 5; i++) {
109             list.add(in.nextInt());     // No index needed! Just .add()
110         }
111
112         /*
113         Case B: You don't know how many items (While Loop) This keeps reading until the input stops (useful for coding contests).
114         To come out of the loop have to press ctrl + d or ctrl + x
115          */
116         while (in.hasNextInt()) {
117             list.add(in.nextInt());
118         }
119
120         /*
121         // OUTPUT SYNTAX
122         Rule: ArrayList has a beautiful built-in toString() method. You don't need a loop at all just to see the data.
123          */
124         System.out.println(list);
125
126
127
128
129         // 2D ArrayList or List of Lists (HIGHLY VALUABLE FOR ATLASSIAN AND CANVA)
130
131         // Step 1: Creating 2D ArrayList
132         ArrayList<ArrayList<Integer>> multiList = new ArrayList<>();
133         /*
134         The "Atlassian" Trap: At this point, you have no rows. If you try to add a number to row 0,
135         the code crashes (IndexOutOfBoundsException).
136          */
137
138         // Step 2: Initialising the adjacency list
```

```
139         // We want 3 rows. We must create 3 empty lists and put them inside the main list.
140         for (int i = 0; i < 3; i++) {
141             multiList.add(new ArrayList<>());
142         }
143         /*
144         Visual before loop: multiList = []
145         Visual after loop:  multiList = [ [], [], [] ] (We now have 3 empty buckets ready for data).
146          */
147 
148 
149         // INPUT SYNTAX
150         // Now that the empty rows exist, you can throw numbers into them.
151 
152         // list.get(i) retrieves the list at index 'i' (the row)
153         // .add(value) adds the number to that specific list
154         multiList.get(0).add(10);
155         multiList.get(0).add(20);
156         multiList.get(0).add(50);
157 
158         //Using Loop for Input (Common in Interviews): If you are reading from a Scanner:
159         for (int i = 0; i < 3; i++) {
160             for (int j = 0; j < 3; j++) {
161                 multiList.get(i).add(in.nextInt()); // Go to row 'i', add the number
162             }
163         }
164 
165 
166         // OUTPUT SYNTAX
167 
168         /*
169         Option A: The "Lazy" Print (Whole Structure) Just like a 1D ArrayList,
170         you can print the whole thing instantly
171          */
172         System.out.println(multiList);
173 
174         /*
175         Option B: Specific Access (Like arr[row][col]) You cannot use square brackets [].
176         You must chain .get() calls.
177          */
178         int val = multiList.get(0).get(1);
179         // 1. list.get(0)   ->  Gets the first list: [10, 20]
180         // 2. .get(1)       ->  Gets index 1 inside that list: 20
181 
182         // Option C: Iterating (Nested Loop)
183         for (ArrayList<Integer> row : multiList) {  // For every list inside the big list...
```

```
184             for (int num : row) {                    // For every
    number inside that small list...
185                 System.out.println(num + " ");
186             }
187             System.out.println();
188         }
189     }
190 
191     /*
192                                             == FAANG Strategy Note ==
193 
194         When you interview at Atlassian or Canva, you will likely
    face a "Graph" problem (e.g., "Find the
195         path between two employees").
196 
197         They will NOT give you the graph as a nice visual. They will
    usually give you a list of edges, and
198         you will have to build this 2D ArrayList yourself to
    represent the connections.
199 
200             - Row 0 represents User 0.
201             - The list inside Row 0 contains all the friends of User
    0.
202 
203         Mastering that "Initialisation Loop (Step 1)" is the first
    like of code for about 30% of all hard
204         interview problems.
205      */
206 }
207 
```